
Équipe 09

PlaniSup
Spécifications des requis du système (SRS)

Version 1.3

Historique des révisions

Date	Version	Description	Auteur
2025-09-04	1.0	Rédaction du document au complet	Équipe 9
2025-09-12	1.1	Correction du document au complet	Équipe 9
2025-09-23	1.2	Correction du document au complet selon la précorrection	Équipe 9
2025-10-02	1.3	Correction final du document pour le 3 octobre 2025	Équipe 9

Table des matières

1. Introduction	3
1.1. But	3
1.2. Vue d'ensemble du document	4
2. Description globale	4
2.1. Caractéristiques des usagers	4
2.2. Interfaces	4
2.2.1. Interfaces usagers	4
2.2.2. Interfaces matérielles	5
2.2.3. Interfaces logicielles	5
2.2.4. Interfaces de communication	5
2.3. Contraintes générales	6
2.4. Hypothèses et dépendances	6
3. Exigences fonctionnelles	7
3.1. Authentification	7
3.2. Gestion du plan d'étude	7
3.3. Validation automatique du plan d'étude	8
3.4. Approbation des plans d'études	8
3.5. Gestion de programme	8
3.6. Recherche de cours	9
3.7. Gestion des membres administratifs	9
4. Exigences non-fonctionnelles	9
4.1. Utilisabilité	9
4.2. Fiabilité	10
4.3. Performance	10
4.4. Maintenabilité	10
4.5. Contraintes de conception	11
4.6. Sécurité	11
4.7. Normes applicables	11
ANNEXE A: Glossaire	12

Spécifications des requis du système (SRS)

1. Introduction

1.1. But

Le SRS décrit le comportement externe de l'application PlaniSup. Il décrit aussi les exigences non-fonctionnelles, les contraintes de conception, ainsi que les autres facteurs nécessaires à la description complète des exigences du logiciel à développer.

1.2. Vue d'ensemble du document

Ce document est structuré pour fournir une compréhension complète des spécifications du système. Il commence par une description globale du logiciel, couvrant les caractéristiques des usagers, les différentes interfaces, les contraintes générales, ainsi que les hypothèses et dépendances. Ensuite, les exigences fonctionnelles sont détaillées, suivies des exigences non-fonctionnelles qui incluent l'utilisabilité, la fiabilité, la performance, la maintenabilité, les contraintes de conception, la sécurité, les exigences de la documentation usager en ligne et du système d'assistance ainsi que les normes applicables. Enfin, un glossaire est fourni en annexe pour clarifier les termes techniques utilisés.

2. Description globale

PlaniSup est une nouvelle application web interactive destinée à la planification, la validation et l'approbation des plans d'études aux cycles supérieurs à Polytechnique Montréal. Elle comprend des modules dédiés aux étudiants, directeurs, agents administratifs, coordonnateurs et administrateurs. L'application permet la construction de parcours personnalisés, assure la validation automatique des plans d'études et prend en charge leur approbation séquentielle. Elle intègre également des fonctionnalités telles que la recherche de cours, la modification du plan d'étude et le suivi interactif du traitement du plan d'étude, tout en reposant sur une architecture flexible, évolutive et conforme aux règlements institutionnels. Tous les modules et fonctionnalités de PlaniSup sont conçus dans le cadre du projet.

2.1. Caractéristiques des usagers

Le projet PlaniSup répond aux besoins des usagers des cycles supérieurs à Polytechnique Montréal. Les étudiants, dont le niveau d'expérience en informatique varie, utilisent l'application pour construire et modifier leur plan d'études. Le développement prend en compte la nécessité d'une interface simple, guidée et intuitive, afin de réduire les erreurs et de faciliter la compréhension des règles de validation. Les directeurs et coordonnateurs, possédant une expérience informatique modérée, doivent approuver les plans d'études ; l'application inclut des fonctionnalités de suivi pour simplifier cette tâche et minimiser les omissions. Les agents administratifs et administrateurs, généralement familiers avec les systèmes institutionnels, assurent la validation et la gestion des plans d'études. L'application prévoit des outils d'administration sécurisés et performants, capables de traiter un grand volume de données et de gérer les droits d'accès de manière précise. L'ensemble de l'application est accessible directement depuis un navigateur web et s'adapte aux différents profils d'utilisateurs tout en respectant les processus institutionnels existants.

2.2. Interfaces

2.2.1. Interfaces usagers

- **Authentification:** Permet à tous les utilisateurs d'accéder à leur compte avec les identifiants institutionnels. L'interface fournit des rétroactions claires en cas d'erreur et propose la récupération de mot de passe. Elle est conçue pour être intuitive, même pour les étudiants ayant peu d'expérience en informatique.
- **Accueil :** Sert de point de départ après connexion. Les étudiants visualisent les options de programmes ou accèdent à la planification de leur plan d'études via des boutons interactifs simples. Les directeurs et coordonnateurs voient la liste des plans d'études soumis par leurs étudiants. Les agents administratifs et administrateurs consultent les plans pour validation et gestion.
- **Planification et consultation du plan d'études :** Permet aux étudiants de construire et modifier leur plan d'études tout en recevant des alertes en temps réel sur les incompatibilités ou problèmes. L'interface est

guidée pour faciliter la compréhension des règles de validation, réduisant les erreurs pour les utilisateurs moins expérimentés.

- **Soumission et suivi** : Permet aux étudiants de soumettre leur plan, d'annuler un plan et de suivre son statut dans le processus de validation. Les commentaires des approbateurs sont accessibles pour assurer la transparence.
- **Gestion et approbation par le personnel académique** : Les directeurs et coordonnateurs disposent d'outils pour approuver ou commenter les plans. Les agents administratifs disposent des mêmes outils pour valider pour commenter le plan d'étude.
- **Administration et coordination des programmes** : Permet de configurer les structures de programmes, gérer les modules, affecter les cours et établir les règles de validation. Les administrateurs peuvent attribuer différents rôles aux différents utilisateurs.

Toutes les interfaces supportent les navigateurs Chrome, Firefox, Edge et Safari. Elles sont réactives et s'adaptent aux résolutions d'écran comprises entre 1366×768 px et 2560×1440 px.

2.2.2. Interfaces matérielles

L'application est disponible pour tous les ordinateurs de bureau ou portables. Il est important de spécifier que les ordinateurs de bureau doivent être munis d'un écran, d'un clavier et d'une souris (ou pavé tactile). L'ordinateur de bureau et le portable doivent être munis d'une carte d'interface réseau pour assurer les communications avec le serveur via Internet. L'application sera hébergée sur un ou plusieurs serveurs standards fournis par le Service informatique de Polytechnique de Montréal.

2.2.3. Interfaces logicielles

L'application est développée avec Angular et Express afin de construire respectivement l'interface utilisateur et le serveur, tous deux étant supportés par le langage TypeScript. L'application intègre également certains services externes. Tout d'abord, l'utilisation de MongoDB comme base de données permet d'assurer la persistance des données. Ensuite, l'intégration du service CAS de Polytechnique, donne la possibilité d'authentifier les utilisateurs de l'application à partir des informations déjà connues et gérées par l'établissement universitaire. Enfin, pour la partie interface utilisateur développée avec Angular, Angular Material est utilisé permettant la création d'interfaces grâce à des composants UI/UX pré conçus et adaptés aux besoins communs des développeurs et des utilisateurs.

2.2.4. Interfaces de communication

L'application est accessible aux clients dès qu'une connexion Internet est disponible, que ce soit via Wi-Fi ou LTE, qui assurent respectivement une connexion sans fil. Au sein de l'architecture logicielle, la communication entre le Frontend (interface utilisateur) et le Backend (serveur) repose sur les protocoles HTTP/HTTPS. Le protocole HTTP permet de faire des requêtes entre les différentes composantes logicielles, tandis que HTTPS garantit la sécurité des requêtes grâce au chiffrement TLS. Ensuite, la gestion des échanges de données est facilitée par l'utilisation d'une API REST, qui définit un ensemble de règles (GET, POST, PUT, PATCH, DELETE) pour permettre la communication entre les composants logiciels. Enfin, les informations transmises circulent principalement sous le format JSON, qui offre une structure légère et manipulable pour échanger les données.

2.3. Contraintes générales

Le projet PlaniSup doit respecter l'ensemble des règlements des études supérieures de Polytechnique Montréal, ce qui implique que toutes les règles de validation des plans d'études telles que le nombre de crédits requis, les cours obligatoires et optionnels et les conditions de prérequis sont codifiées dans le système. Cette contrainte impacte directement les fonctionnalités de planification et de pré-validation en temps réel pour les étudiants. Le système doit également gérer les particularités des sept départements et des deux instituts, ce qui nécessite une configuration dynamique des programmes, modules et règles de validation, ainsi qu'une interface d'administration flexible pour permettre aux administrateurs et coordonnateurs de paramétrer les spécificités de chaque unité.

PlaniSup doit supporter différents types de programmes, incluant les structures modulaires des maîtrises professionnelles, les exemptions au doctorat et le parcours Avantage Poly. Cela implique que les fonctionnalités de planification et de validation tiennent compte de ces variations, notamment pour le calcul des crédits, la gestion des exemptions et la compatibilité avec les parcours spécifiques. La sécurité constitue une contrainte majeure : seules les personnes disposant d'un compte institutionnel peuvent accéder à l'application, ce qui impose un système d'authentification unique via les identifiants Polytechnique et des rôles clairement définis pour chaque type d'utilisateur (étudiant, directeur, agent administratif, administrateur).

Le projet doit également garantir des performances satisfaisantes même avec un grand volume de plans d'études. Cette exigence impacte l'architecture technique et la conception de la base de données afin de maintenir des temps de réponse faibles et assurer la fluidité des interfaces. Par ailleurs, l'application doit s'intégrer aux systèmes existants, en particulier pour l'import des listes de cours et la synchronisation des informations des étudiants, ce qui nécessite l'utilisation d'API sécurisées et le respect des droits d'accès externes. Les normes de codage et de maintenance représentent une autre contrainte : le code doit respecter les conventions institutionnelles, être documenté et structuré pour faciliter la testabilité, la scalabilité et les modifications futures.

Dans le cadre de ce projet, le Minimum Viable Product (MVP) inclut l'authentification des utilisateurs et la gestion des rôles, la construction et la pré-validation des plans d'études par les étudiants, la soumission et le suivi des plans, ainsi que les interfaces de validation pour les directeurs et agents administratifs, tout en respectant les règles de validation propres aux programmes et aux départements.

2.4. Hypothèses et dépendances

Dépendances :

- **Connexion Internet** : Il est impératif que les utilisateurs soient connectés à Internet afin d'accéder à l'application PlaniSup. Sans connexion, les étudiants ne peuvent soumettre leur plan d'études dans le système et les autres membres du personnel universitaire impliqués dans le processus de validation ne peuvent consulter les plans transmis par les demandeurs.
- **Navigateurs web** : Il est impératif que les utilisateurs emploient des navigateurs courants tels que Chrome, Firefox, Edge ou Safari pour accéder à l'application. Dans le cas contraire, certains éléments de l'interface utilisateur peuvent ne pas s'afficher correctement, entraînant ainsi des incohérences et des erreurs avec le comportement initialement attendu.
- **Serveurs de Polytechnique** : Il est impératif que les serveurs de Polytechnique soient opérationnels afin d'assurer le bon fonctionnement de l'application. Comme le serveur applicatif et la base de données sont hébergés sur l'infrastructure de l'établissement, toute indisponibilité de ces ressources rend PlaniSup inaccessible aux utilisateurs. Une défaillance peut également entraîner des erreurs dans le transfert des données ainsi que des risques liés à la sécurité de l'application et des informations qu'elle traite.

- **Services externes utilisés :** Il est impératif que les services externes intégrés à l'application soient continuellement disponibles. À titre d'exemple, il est nécessaire que l'outil MongoDB qui gère la base de données de PlaniSup soit opérationnel en tout temps. Dans le cas contraire, les données relatives aux cours, aux programmes et aux plans d'études deviennent inaccessibles, rendant l'application inutilisable. Il est donc essentiel de s'appuyer sur un service fiable, tel que MongoDB.

Hypothèses :

- **Service CAS de Polytechnique :** Le service CAS de Polytechnique est opérationnel et permet aux utilisateurs de se connecter à leur compte universitaire ainsi que d'accéder aux rôles qui leur sont associés (étudiant, directeur, agent administratif, coordonnateur ou administrateur).
- **Processus de validation des plans d'études :** Le processus de validation des plans d'études est déjà connu du personnel universitaire. Lorsqu'un étudiant soumet son plan d'études, chaque membre du personnel concerné sait à quel moment il doit effectuer la vérification et la validation avant de laisser la procédure se poursuivre.
- **Connaissance des cours déjà accomplis par l'étudiant :** L'étudiant qui soumet son plan d'études dispose d'une connaissance préalable des cours qu'il a déjà réussis lors de ses études antérieures à Polytechnique. Cette connaissance lui permet de sélectionner correctement le programme de cycles supérieurs qu'il souhaite entreprendre et d'éviter de choisir des cours déjà complétés. Il connaît également le directeur et le coordonnateur à qui il doit soumettre le plan d'étude.

3. Exigences fonctionnelles

3.1. Authentification

<i>Identificateur</i>	<i>Exigence</i>	<i>Type d'exigence</i>	<i>Type d'utilisateur</i>
3.1.1	Le système doit permettre à l'utilisateur PlaniSup de s'authentifier à son compte utilisateur.	Essentielle	Utilisateur PlaniSup
3.1.2	Le système doit créer un nouveau compte utilisateur lors de la première connexion d'un utilisateur.	Essentielle	
3.1.3	Le système doit rediriger l'utilisateur vers son compte existant à partir de la deuxième authentification.	Essentielle	
3.1.4	Le système doit créer un compte de type employé pour l'utilisateur dont le code permanent commence par «p» .	Essentielle	Membre Administratif
3.1.5	Le système doit utiliser le service d'authentification CAS de Polytechnique pour authentifier un utilisateur.	Souhaitable	

3.2. Gestion du plan d'étude

<i>Identificateur</i>	<i>Exigence</i>	<i>Type d'exigence</i>	<i>Type d'utilisateur</i>
3.2.1	Le système doit permettre à l'étudiant de créer son plan d'étude.	Essentielle	Étudiant
3.2.2	Le système doit permettre à l'étudiant de modifier les informations de plan d'étude.	Essentielle	Étudiant
3.2.3	Le système doit permettre à l'étudiant de visualiser son plan d'étude.	Essentielle	Étudiant
3.2.4	Le système doit présenter les cours disponibles dans le programme.	Essentielle	Étudiant
3.2.5	Le système doit permettre à l'étudiant de soumettre son plan d'étude au processus administratif de validation.	Essentielle	Étudiant
3.2.6	Le système doit permettre à l'étudiant d'abandonner son plan d'étude.	Essentielle	Étudiant

3.2.7	Le système doit permettre de consulter une rétroaction d'un membre administratif du processus de validation.	Essentielle	Étudiant
3.2.8	Le système doit permettre à l'étudiant de consulter l'état d'avancement de sa demande d'approbation de plan d'étude.	Essentielle	Étudiant

3.3. Validation automatique du plan d'étude

<i>Identificateur</i>	<i>Exigence</i>	<i>Type d'exigence</i>	<i>Type d'utilisateur</i>
3.3.1	Le système doit vérifier automatiquement le nombre total de crédits du plan d'étude.	Essentielle	Étudiant
3.3.2	Le système doit vérifier la répartition correcte du nombre total de crédits par module.	Essentielle	Étudiant
3.3.3	Le système doit vérifier la présence des cours obligatoires.	Essentielle	Étudiant
3.3.4	Le système doit détecter l'excès de crédits hors programme.	Essentielle	Étudiant
3.3.5	Le système doit signaler une incompatibilité entre les cours.	Essentielle	Étudiant
3.3.6	Le système doit signaler une incompatibilité entre les modules.	Essentielle	Étudiant
3.3.7	Le système doit générer une rétroaction détaillée des règles non respectées du plan d'étude.	Essentielle	Étudiant

3.4. Approbation des plans d'études

<i>Identificateur</i>	<i>Exigence</i>	<i>Type d'exigence</i>	<i>Type d'utilisateur</i>
3.4.1	Le système doit afficher un plan d'étude en attente de validation.	Essentielle	Membre Administratif
3.4.2	Le système doit permettre à un membre administratif de fournir une rétroaction sur le plan d'étude.	Essentielle	Membre Administratif
3.4.3	Le système doit permettre à un membre administratif de valider un plan d'étude.	Essentielle	Membre Administratif

3.5. Gestion de programme

<i>Identificateur</i>	<i>Exigence</i>	<i>Type d'exigence</i>	<i>Type d'utilisateur</i>
3.5.1	Le système doit permettre à l'administrateur de créer un programme.	Essentielle	Administrateur
3.5.2	Le système doit permettre à l'administrateur de modifier l'information d'un programme.	Essentielle	Administrateur
3.5.3	Le système doit permettre à l'administrateur de supprimer un programme.	Essentielle	Administrateur

3.6. Recherche de cours

<i>Identificateur</i>	<i>Exigence</i>	<i>Type d'exigence</i>	<i>Type d'utilisateur</i>
3.6.1	Le système doit proposer un moteur de recherche de cours avec filtrage par code.	Essentielle	Tous
3.6.2	Le système doit proposer un moteur de recherche de cours avec filtrage par titre.	Essentielle	Tous
3.6.3	Le système doit proposer un moteur de recherche de cours avec filtrage par thème.	Essentielle	Tous
3.6.4	Le système doit proposer un moteur de recherche de cours avec filtrage par langue.	Essentielle	Tous
3.6.5	Le système doit proposer un moteur de recherche de cours avec filtrage par session.	Essentielle	Tous

3.7. Gestion des membres administratifs

<i>Identificateur</i>	<i>Exigence</i>	<i>Type d'exigence</i>	<i>Type d'utilisateur</i>
3.7.1	Le système doit afficher l'utilisateur dont le type de compte est employé.	Essentielle	Administrateur
3.7.2	Le système doit permettre à l'administrateur d'attribuer le rôle de directeur à un compte employé.	Essentielle	Administrateur
3.7.3	Le système doit permettre à l'administrateur d'attribuer le rôle d'agent administratif à un compte employé.	Essentielle	Administrateur
3.7.4	Le système doit permettre à l'administrateur d'attribuer le rôle de coordonnateur à un compte employé.	Essentielle	Administrateur
3.7.5	Le système doit permettre à l'administrateur d'attribuer le rôle de registraire à un compte employé.	Essentielle	Administrateur

4. Exigences non-fonctionnelles

4.1. Utilisabilité

<i>Identificateur</i>	<i>Description</i>
4.1.1	Le système doit permettre à un étudiant de créer un plan d'étude en moins de 5 minutes.
4.1.2	Le système doit permettre à un étudiant de soumettre un plan d'étude en moins de 10 secondes.
4.1.3	Le système doit permettre à un étudiant de modifier un plan d'étude en moins de 5 minutes.
4.1.4	Le système doit permettre à un membre administratif de valider un plan d'étude après une formation d'au plus 5 minutes.
4.1.5	Le système doit permettre à un membre administratif de rejeter un plan d'étude après une formation d'au plus 5 minutes.
4.1.6	Le système doit permettre à un administrateur d'ajouter un programme après une formation d'au plus 5 minutes.
4.1.7	Le système doit permettre à un administrateur de modifier un programme après une formation d'au plus 5 minutes.

4.1.8	Le système doit permettre à un administrateur de supprimer un programme après une formation d'au plus 1 minute.
4.1.9	Le système doit permettre à un étudiant de repérer une erreur dans son plan d'étude en moins de 10 secondes.
4.1.10	Le système doit permettre à un utilisateur de se connecter à son compte Planisup en moins de 5 secondes.
4.1.11	Le système doit permettre à un utilisateur de rechercher un cours en moins de 15 secondes.

4.2. Fiabilité

<i>Identificateur</i>	<i>Description</i>
4.2.1	Le système doit être disponible au moins 99,9 % du temps.
4.2.2	Le système doit avoir un temps moyen entre pannes d'au moins 3 mois.
4.2.3	Le système doit être rétabli en moins de 6 heures en cas de panne.

4.3. Performance

<i>Identificateur</i>	<i>Description</i>
4.3.1	Le système doit être capable d'afficher le plan d'étude avec un temps de réponse de moins de 30 secondes.
4.3.2	Le système doit mettre à jour le plan d'étude avec un temps de réponse de moins de 30 secondes.
4.3.3	Le système doit être capable de rechercher un cours avec un temps de réponse de moins de 5 secondes.
4.3.4	Le système doit être capable de valider automatiquement un plan d'étude avec un temps de réponse de moins 5 secondes.
4.3.5	Le système doit être capable de supporter au moins 500 utilisateurs connectés simultanément sans dégradation notable des performances.
4.3.6	Le système doit être capable de naviguer d'une section à une autre de l'application avec un temps de réponse de moins de 1 seconde.

4.4. Maintenabilité

<i>Identificateur</i>	<i>Description</i>
4.4.1	Le système doit posséder une architecture modulaire pour permettre une mise à jour sans affecter le reste de l'application.
4.4.2	Le système doit inclure une documentation claire destinée à un futur développeur.
4.4.3	Le système doit posséder un code source pour assurer le suivi d'une tâche.
4.4.4	Le système doit posséder un code source pour assurer le suivi d'un bogue.
4.4.5	Le système doit posséder un code source respectant une norme de codage de type CamelCase.

4.5. Contraintes de conception

<i>Identificateur</i>	<i>Description</i>
4.5.1	Le système doit être développé à l'aide du cadre Angular pour l'interface utilisateur.
4.5.2	Le système doit être développé à l'aide de NodeJS pour le serveur.
4.5.3	Le système doit être développé à l'aide de MongoDB pour la base de données.
4.5.4	Le système doit être hébergé sur le serveur de Polytechnique Montréal.
4.5.5	Le système doit respecter la politique de sécurité de Polytechnique Montréal.
4.5.6	Le système doit respecter la politique d'infrastructure de Polytechnique Montréal.
4.5.7	Le système doit authentifier un utilisateur via le système d'authentification CAS de Polytechnique.
4.5.8	Le système doit pouvoir être exécuté sur un ordinateur quelconque.

4.6. Sécurité

<i>Identificateur</i>	<i>Description</i>
4.6.1	Le système doit autoriser uniquement une connexion authentifiée.
4.6.2	Le système doit sécuriser une communication à l'aide du chiffrement du protocole HTTPS.

4.7. Normes applicables

<i>Identificateur</i>	<i>Description</i>
4.7.1	Le système doit se conformer aux politiques de Polytechnique Montréal concernant la protection des données personnelles.
4.7.2	Le système doit se conformer aux politiques de Polytechnique Montréal concernant la sécurité de l'information.
4.7.3	Le système doit assurer une qualité du code à l'aide de revues de code tout au long du projet.

ANNEXE A: Glossaire

Mots, acronymes, abréviations	Définition
Angular	Cadriciel utilisé pour l'application web .
API	Interface de programmation applicative (Application Programming Interface).
CamelCase	CamelCase est une convention de dénomination utilisée en programmation et en informatique pour écrire des mots composés sans espaces, en capitalisant la première lettre de chaque mot sauf le premier.
CPES	Coordonnateur des Programmes d'Études Supérieures.
État d'avancement	L'étape à laquelle le processus de validation est rendu: directeur, agent administratif, coordonnateur et registraire.
GitLab	GitLab est une plateforme DevOps complète qui intègre la gestion de projet, le suivi des problèmes, le contrôle de version, et l'automatisation des pipelines CI/CD.
HTTP	HTTP (HyperText Transfer Protocol) est un protocole de communication utilisé pour échanger des informations entre un client et un serveur.
Informations de plan d'études	Les informations du plan d'études comprennent les modules du programme, la liste des cours avec leur sigle, le trimestre associé, l'indication si le cours a déjà été suivi ainsi que la note obtenue.
Informations d'un programme	Les informations d'un programme comprennent les modules du programme, la liste des cours permis avec leur sigle et les règles de validation du programme.
Membre Administratif	Directeur, agent administratif, coordonnateur et registraire.
MongoDB	Application pour héberger la base de données.
MTBF	Temps moyen de réparation (Mean Time Between Failures)
MVP	Produit minimum viable (Minimum Viable Product)
NodeJS	Node.js est un environnement d'exécution JavaScript côté serveur.
Processus administratif de validation	Processus séquentiel dans lequel le plan d'étude est validé par les membres administratifs de Polytechnique (Directeur, Agent administratif, Coordonnateur et Registraire).
Programme	Maîtrise, DESS ou doctorat.
Px	Unité représentant les pixels.
Règles de validation du programme	Les règles du programme sont les cours admissibles, le nombre de crédits à respecter et d'autres règles plus complexes comme l'utilisation des cours non choisis d'un module ultérieur.
Rétroaction des membres administratifs	Commentaires écrits reliés à la correction d'un plan d'étude.
TypeScript	Langage de programmation utilisé pour l'application.
Utilisateur PlaniSup	Les six différents rôles présents au sein de l'application PlaniSup. On y retrouve l'étudiant, le directeur, l'agent administratif, le coordonnateur, le registraire et l'administrateur.